FIAP

Vetores Numéricos (Arrays) e Vetores de Caracteres (Strings) em C

Ciência da Computação

Prof. Dr. Érick Toshio Yamamoto

Objetivos da Aula - Parte 1

FIAP

O que vamos aprender hoje:

- Entender o conceito de **arrays** e armazenamento contíguo.
- Aprender a **declarar, inicializar e acessar** elementos.
- Percorrer vetores de forma eficiente usando laços **for**.
- Praticar com uma **atividade em sala de aula** .

Foco: Manipulação de Dados em Lote com Vetores

Objetivos da Aula - Parte 2

FIAP

O que vamos aprender hoje:

- Compreender **strings** como vetores de caracteres.
- Entender o papel vital do **terminador nulo (\0)** .
- Aprender a ler e escrever strings com **segurança** .
- Usar a biblioteca **string.h** para manipulação avançada.

Foco: Processamento de Texto e Comunicação com o Usuário

Por que precisamos de vetores?

FIAP

```c
// Como armazenar 50 notas?
float nota1, nota2, nota3, nota4, nota5...;

printf("Nota 1: ");
scanf("%f", ¬a1);
printf("Nota 2: ");
scanf("%f", ¬a2);
// ... Repetir 50 vezes?

float media = (nota1 + nota2 + ... + nota50) /
50;
```

- **Código Extenso:** Declarar dezenas de variáveis isoladas polui o código.
- **Impossível Automatizar:** Você não consegue usar laços de repetição com nomes de variáveis diferentes.
- **Manutenção Difícil:** Alterar a quantidade de dados exige mudar todo o código.
- **Solução:** Agrupar todos os dados sob um único nome e acessá-los por um índice.

Vetor: Um Conjunto de Dados Homogêneos

FIAP

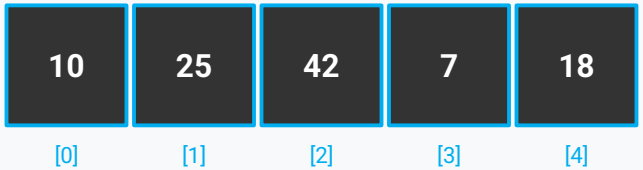
Homogeneidade: Uma estrutura que armazena múltiplos valores do **mesmo tipo** (ex: todos int).

Armazenamento Contíguo: Os dados são guardados em endereços de memória vizinhos.

Identificação: Cada posição é acessada através de um **índice** numérico.

> "Um vetor é como um prédio onde cada apartamento tem o mesmo tamanho e um número único."

Representação na Memória



Espaço contínuo de memória reservado para o tipo declarado.

Onde tudo começa: O Índice Zero

FIAP

- Em C, o primeiro elemento de um vetor está sempre no **índice 0**.
- Se um vetor tem tamanho **N**, o último índice válido é **N-1**.
- O compilador C não verifica limites de array em tempo de execução.

Cuidado: *Acessar* `vetor[N]` *resultará em comportamento indefinido ou erro de segmentação.*

Representação na Memória (Tamanho 5)



O índice indica o **deslocamento** em relação ao início.

Criando e Alimentando Vetores

FIAP

</> Sintaxe de Declaração

- tipo nome[tamanho];
- O **tamanho** deve ser um valor constante e inteiro.
- O **tipo** define o que cada posição do vetor irá armazenar.

Importante: *Se não inicializar, o vetor conterá valores residuais ("lixo de memória").*

```
// Declaracao simples
int notas[5];
 // Declaracao com inicializacao
int pares[3] = {2, 4, 6};
 // Inicializacao parcial (restante vira 0)
float valores[5] = {10.5, 20.0};
 // Tamanho omitido (compilador define)
int idades[] = {18, 20, 25};
```

Lendo e Escrevendo no Vetor

FIAP

Escrita: Atribuímos valores usando o índice. Ex: notas[0] = 10;

Leitura: Acessamos o valor para exibir ou calcular. Ex: printf("%d", notas[2]);

Cada posição se comporta exatamente como uma **variável individual** do tipo declarado.

Importante: *O índice deve ser sempre um valor inteiro entre* *0* *e* *N-1* *.*

```c
int idades[3];
 // Escrita de dados
idades[0] = 20;
idades[1] = 25;
idades[2] = 30;
 // Leitura de dados
printf("Primeira idade: %d\n", idades[0]);
 // Uso em calculos
int soma = idades[0] + idades[1];
```

Automatização com Laços de Repetição

FIAP

- O laço **for** é a ferramenta ideal para percorrer vetores de forma sistemática.
- Usamos a variável de controle do laço (ex: `i`) como o **índice** do vetor.
- Isso permite processar centenas de dados com apenas algumas linhas de código.

> ***Regra:*** *O laço deve ir de* `0` *até* `tamanho - 1` .

```c
int numeros[5];
int i;
 // Lendo os dados
for(i = 0; i < 5; i++) {
scanf("%d", &numeros[i]);
}
 // Imprimindo os dados
for(i = 0; i < 5; i++) {
printf("%d ", numeros[i]);
}
```

Algoritmos Básicos com Vetores

FIAP

- **Acúmulo:** Usamos uma variável (ex: `soma`) para somar todos os elementos do vetor.
- **Média:** Após o laço, dividimos o total pelo número de elementos.
- Técnica fundamental para processamento de grandes volumes de dados.

> ***Atenção:*** *Sempre inicialize a variável acumuladora com `0` antes do laço.*

```
float notas[5] = {7.5, 8.0, 6.5, 9.0, 10.0};
float soma = 0, media;
int i;
 // Somando os elementos
for(i = 0; i < 5; i++) {
soma += notas[i];
}
 // Calculando a media
media = soma / 5;
 printf("Media: %.2f", media);
```

Algoritmos de Comparação

FIAP

Estratégia: Inicializamos a variável `maior` com o primeiro elemento do vetor (`v[0]`).

Laço: Percorremos o vetor comparando cada elemento com o `maior` atual.

Atualização: Se o elemento atual for maior, ele passa a ser o novo `maior`.

> ***Dica:*** *A mesma lógica se aplica para encontrar o* ***menor*** *valor, apenas invertendo o sinal da comparação.*

```
int v[5] = {10, 25, 42, 7, 18};
int maior = v[0];
int i;
 for(i = 1; i < 5; i++) {
if(v[i] > maior) {
maior = v[i];
}
}
 printf("O maior valor e: %d", maior);
```

Praticar Agora: Gestão de Notas

FIAP

Desenvolva um programa que:

- Crie um vetor para armazenar **5 notas** de um aluno.
- Use um laço `for` para ler as notas do teclado.
- Calcule a **média aritmética** dessas notas.
- Ao final, percorra o vetor novamente e imprima quantas notas estão **acima da média**.

Objetivo: Praticar a declaração, leitura e o processamento de dados em lote usando vetores e laços de repetição.

Strings: Além dos Números

FIAP

- Em C, **não existe** um tipo de dado "String" nativo (como em Java ou Python).
- Usamos **vetores de caracteres** para representar texto: `char nome[20];`
- Uma string é apenas uma sequência de bytes na memória interpretados como caracteres ASCII.

> "Tudo o que você digita é, no fundo, um vetor de pequenos inteiros (caracteres)."

Estrutura de uma String



Cada letra ocupa 1 byte na memória.
O final é marcado por um caractere especial.

O Segredo das Strings: O \0

FIAP

- O C precisa saber onde a palavra termina. O caractere **\0** (nulo) marca o fim da string.
- **Regra de Ouro:** O vetor deve ter tamanho **N+1** para armazenar uma palavra de **N** letras.
- Sem o \0 , as funções de string continuarão lendo a memória até encontrar um zero, causando erros.

Exemplo: *Para guardar "CASA" (4 letras), precisamos de um* *char[5]* .

String "CASA" na Memória



O compilador reserva o último espaço automaticamente em inicializações diretas.

Formas de Criar Strings

FIAP

Aspas Duplas: O C coloca o terminador \0 automaticamente ao final.

Tamanho Omitido: O compilador reserva o espaço exato (letras + 1).

Caractere por Caractere: Exige que você coloque o \0 manualmente.

> ***Dica:*** *Sempre reserve um espaço a mais do que o número máximo de letras esperado.*

```c
// Forma mais comum
char s1[10] = "Aula";
 // Tamanho definido pelo compilador (5)
char s2[] = "FIAP";
 // Forma explicita (pouco usada)
char s3[3] = {'O', 'i', '\0'};
 // Apenas declaracao
char nome[50];
```

Como ler texto do usuário?

FIAP

⚠ **scanf("%s", s):** Para no primeiro espaço. Não limita o tamanho (risco de *Buffer Overflow*).

✅ **fgets(s, tam, stdin):** Lê espaços e respeita o limite do vetor. É a forma **profissional**.

ℹ O `fgets` também captura o caractere de nova linha (`\n`) ao pressionar Enter.

> ***Dica:*** *Nunca use `gets()` . Ela foi removida do padrão C por ser extremamente insegura.*

Inseguro (scanf):

```c
scanf("%s", nome); // Se digitar "Joao Silva", le apenas "Joao"
```

Seguro (fgets):

```c
// Le ate 19 letras + \0
fgets(nome, 20, stdin);
// Remove o \n se necessario
nome[strcspn(nome, "\n")] = 0;
```

Exibindo Texto na Tela

FIAP

printf()

- Usa o formatador `%s`.
- Altamente flexível (permite misturar tipos).
- Não pula linha automaticamente.

puts()

- Mais simples e rápida.
- Imprime a string e **pula uma linha** (\n).
- Ideal para mensagens simples.

```c
char msg[] = "Ola, Aluno!";
 // Usando printf
printf("Mensagem: %s\n", msg);
 // Usando puts
puts(msg);
puts("Bem-vindo a aula de C");
```

Dica: *Use `printf` para relatórios formatados e `puts` para avisos rápidos.*

A Biblioteca <string.h>

FIAP

```
#include <string.h>
```

Ferramentas Prontas: Contém funções otimizadas para as tarefas mais comuns com texto.

Produtividade: Evita que você precise reinventar a roda para medir ou copiar strings.

Padronização: Segue o padrão ANSI C, garantindo que seu código funcione em qualquer compilador.

Função	O que faz?
`strlen()`	Retorna o tamanho da string.
`strcpy()`	Copia uma string para outra.
`strcat()`	Concatena (junta) duas strings.
`strcmp()`	Compara se duas strings são iguais.

Medindo e Copiando Strings

FIAP

strlen()

- Retorna a quantidade de caracteres úteis.
- **Não conta** o terminador nulo `\0`.

strcpy(destino, origem)

- Copia o conteúdo da origem para o destino.
- **Importante:** Substitui o que havia antes.

> ***Aviso:*** *Em C, você* ***não pode*** *fazer* `s1 = s2;` *para copiar strings. Use sempre* `strcpy`

```c
char s1[20];
char s2[] = "Programacao";
int tam;
 // Medindo o tamanho
tam = strlen(s2); // Retorna 11
 // Copiando a string
strcpy(s1, s2);
 printf("S1 agora e: %s\n", s1);
printf("Tamanho: %d", tam);
```

Concatenando e Comparando

FIAP

strcat(destino, origem)

- Anexa a string de origem ao final da string de destino.
- Certifique-se que o destino tem espaço suficiente!

= strcmp(s1, s2)

- Compara duas strings caractere por caractere.
- Retorna **0** se forem idênticas.

> ***Cuidado:*** *Em C,* `s1 == s2` *compara o* ***endereço de memória*** *, não o conteúdo do texto.*

```c
char s1[20] = "Bom ";
char s2[] = "dia!";
 // Concatenando
strcat(s1, s2); // s1 vira "Bom dia!"
 // Comparando
if(strcmp(s1, "Bom dia!") == 0) {
puts("Strings iguais!");
} else {
puts("Diferentes!");
}
```

Praticar em Casa: Sistema de Login

FIAP

Desenvolva um programa que:

- Defina um **usuário** e uma **senha** padrão (hardcoded) usando strings.
- Solicite ao usuário que digite seu login e senha usando `fgets` .
- Use `strcmp` para validar se os dados digitados são iguais aos padrões.
- Exiba "Acesso Permitido" ou "Acesso Negado" conforme o resultado.

Objetivo: Praticar a leitura segura de texto e a comparação lógica de strings em um cenário real de aplicação.